

Programming Assignment 3

Filename: pa3.c

You must include the following commented lines at the beginning of your code to declare authorship. Submissions that do not include these lines will receive a 10% deduction.

```
/* COP 3502C PA3  
This program is written by: Your Full Name */
```

In this activity, you will practice designing, implementing and utilizing data structures in C, focusing on the following:

- Implementing and manipulating singly linked lists to model a dynamic waiting queue
- Simulating a discrete-time system involving two service providers
- Working with structures, pointers, and pointer reassignment in a queue-based context

Background

The *Whisker Wellness Clinic* operates for exactly 8 hours (480 minutes) each day. Two veterinarians, Dr. *Uno* and Dr. *Dos*, are available throughout the clinic day. Cats arrive throughout the day and wait in a shared queue to receive treatment. Unfortunately, a chemical leak was discovered in Dr. *Dos*'s treatment room. As a result, the clinic must conduct contact tracing to notify the owners of all cats treated by Dr. *Dos*.

However, the clinic's visit log for that day became corrupted. Arrival times were originally listed in chronological order, but due to corruption, this ordering is no longer guaranteed. Your task is to reconstruct the events of the day and determine:

- Which cats were successfully treated
- Which cats could not be accommodated
- Which cats were treated by Dr. *Dos* (for contact tracing)

Strategy

One approach to solving this problem is to use a **minute-by-minute simulation**. Maintain a time counter that runs from 0 to 479. At each minute:

- Check whether any new cats arrive and add them to the waiting queue.
- Determine whether each doctor is currently treating a cat or is available.

This method makes it easy to track doctor availability and assign waiting cats as soon as a doctor becomes free. Refer to the sample visualization at the end of this document for guidance.

Simulation Rules

Doctor Assignment Rules

Both doctors are available at time 0. If a doctor is free and the next cat in the queue has already arrived:

- Assign the cat immediately.
- Treatment begins at that minute.
- The doctor becomes busy for `duration` minutes.

Priority Rule

All cats wait in a single shared queue. The queue follows a strict FIFO (First-In, First-Out) policy, so cats are treated in the exact order they arrive. If both doctors become available at the same time, the next waiting cat must be assigned to Dr. Uno first.

Time Constraint

A cat can be accommodated only if its entire treatment is completed before the clinic closes. Specifically, the treatment must begin before minute 480 and must finish at or before minute 480. Formally, a cat can be treated only if:

$$\text{start_time} + \text{duration} \leq 480$$

Exposure Rule

A cat is considered exposed if it was treated by Dr. Dos at any time during the day.

Zero-Duration Rule

If a cat's duration is 0, this means the cat left the clinic immediately without receiving any treatment. Such a cat does not occupy a doctor's time and is not treated. This cat should **not** appear in the treatment log, accommodation list, or exposure report.

Required Data Structures

You must use the following structures in this assignment. You are allowed to declare more structures if needed. However, you are not allowed to change the required structures.

```
typedef struct Cat_s {
    int arrival;           // Cat's arrival time at the clinic (unique)
    char *name;            // Cat's name (unique)
    int duration;          // Required treatment time in minutes
} Cat;

typedef struct SLLNode_s {
    Cat *cat;              // Pointer to a dynamically allocated Cat
    struct SLLNode_s *next;
} SLLNode;
```

Starter Code

Your task is to complete the given C program. You can access the starter code [here](#).

Input

The program reads input from a file named `logs.txt`. The file contains an arbitrary number of input lines in the following format:

```
<arrival_time> <name> <duration>
```

Where:

- `arrival_time` is a nonnegative integer representing the minute the cat arrived.
- `name` is a string (maximum length 25 characters) representing the cat's name and must be dynamically allocated.
- `duration` is a nonnegative integer representing the required treatment time (in minutes).
- Arrival times are guaranteed to be unique (no two cats arrive at the same time).
- Cat names are guaranteed to be unique.

Input terminates when a single line containing only `-1` is encountered.

Important Note

You may assume that all input strings contain no whitespace.

Output

For each successful treatment (in chronological simulation order), print:

```
Doctor X treated <name> at <time>
```

where `X` is either `Uno` or `Dos`, and `<time>` is the minute the treatment began.

After printing all successful treatments, print any cats that could not be accommodated (if any) in the following format:

```
Cannot accommodate <name>
```

Rejected cats must be printed in the chronological order in which they are determined to be unable to receive treatment during the simulation (i.e., cannot accommodate due to the clinic closing).

Finally, print the exposure report. If one or more cats were treated by Dr. Dos, print the names of exposed cats in reverse chronological order of treatment (i.e., most recently treated first) in the following format:

```
Exposed Cats
<name>
<name>
...
```

If no cats were treated by Dr. Dos, print:

```
No Exposed Cats
```

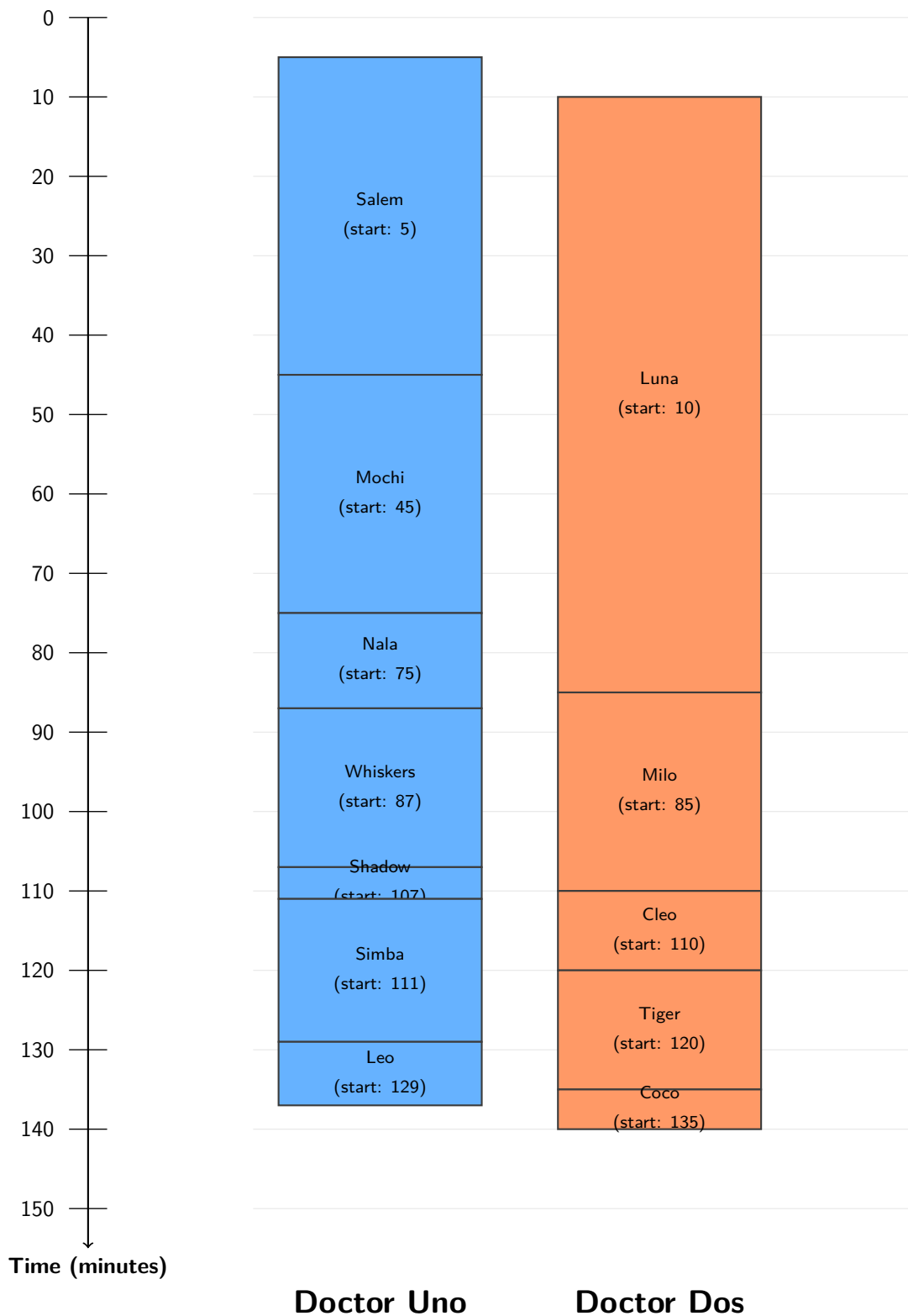
Sample Input logs.txt

```
25 Whiskers 20
10 Luna 75
60 Tiger 15
11 Mochi 30
33 Shadow 4
14 Puff 0
5 Salem 40
44 Cleo 10
18 Milo 25
70 Leo 8
13 Nala 12
52 Simba 18
90 Coco 5
475 Midnight 10
-1
```

Sample Output

```
Doctor Uno treated Salem at 5
Doctor Dos treated Luna at 10
Doctor Uno treated Mochi at 45
Doctor Uno treated Nala at 75
Doctor Dos treated Milo at 85
Doctor Uno treated Whiskers at 87
Doctor Uno treated Shadow at 107
Doctor Dos treated Cleo at 110
Doctor Uno treated Simba at 111
Doctor Dos treated Tiger at 120
Doctor Uno treated Leo at 129
Doctor Dos treated Coco at 135
Cannot accommodate Midnight
Exposed Cats
Coco
Tiger
Cleo
Milo
Luna
```

The image below is a partial visualization of the expected output. It displays only the cats that received treatment. The vertical axis represents the elapsed time since the clinic opened. In the full output, this line should extend to 480 minutes; however, for brevity, the figure shown here is only up to 150.



Additional Implementation Requirements

- Your solution must utilize a linked list. Otherwise, you will receive a score of 0.
- You are not allowed to use any sorting algorithm or any built-in sort function. Such approach will result in a score of 0.
- Your solution may not include any global variables.
- For strings, you must first read the input into a static char array, then allocate the exact amount of memory needed based on its length and copy the string using strcpy function.
- You must free all the memory to receive full credit.
- Your code must have no memory leaks at the end of its execution.

Hints

- When reconstructing the chronological order of the queue, perform ordered insertion at the time of insertion. You are not allowed to sort the list after it has been built. Each new node must be placed immediately into its proper position according to arrival time.
- Define appropriate structures to help you define the queue and the stack.
- Consider writing helper functions to manage the queue and the stack.
- Test with small inputs before trying larger cases.

Clarifications

For questions or clarifications about the program requirements, please post them in the corresponding discussion thread on Webcourses. If additional clarification is needed, you are welcome to visit the TAs or the instructor during office hours. Requests for help with debugging should be brought to office hours rather than sent by email.

Tentative Rubric (Subject to Change)

A set of test cases will be used to evaluate whether your program produces the expected output. Each failed test case will result in a grade deduction per the rubric below. In addition to the sample test cases, we will use additional (hidden) test cases during grading. Therefore, passing the sample test cases does not guarantee full credit. You are strongly encouraged to thoroughly test your code against a variety of inputs. Because some test cases are hidden, your final score will not be visible until the submission has been fully graded.

Important: Any attempt to hard-code the expected output or otherwise circumvent the grading process constitutes a violation of academic integrity and will be handled in accordance with university policy.

- If your code does not compile on Gradescope, you will receive a score of 0.
- If you modify, omit, or fail to use the required data structures, you may receive a score of 0.
- Failure to use dynamic memory allocation for storing data will result in a score of 0.
- Correct implementation of all functions and specifications: 30%.
- Properly freeing all dynamically allocated memory with zero memory leaks: 10%.
- Passing all test cases with exact output matching: 60%.